

Dokumentacja projektu - Baza medycznych badań obrazowych

Desktopowa aplikacja do przechowywania, wyszukiwania i przeglądania medycznych badań obrazowych (RTG, CT, USG, MRI) w formacie DICOM. Napisana w C++ z biblioteką Qt, baza danych PostgreSQL przez ODBC, obsługa plików DICOM przez bibliotekę DCMTK.

Repozytorium: https://github.com/mikita-salauyou/SIM_MedicalDatabase

1. Opis funkcjonalny

Aplikacja realizuje następujące funkcje:

- logowanie do bazy danych (okno logowania, połączenie przez ODBC),
- rejestr pacjentów (dodawanie, edycja, usuwanie, wyszukiwanie po nazwisku),
- ewidencja badań przypisanych do pacjenta (data, typ, opis),
- import pojedynczych plików DICOM oraz całych serii (folderów),
- automatyczne wykrywanie modalności (CT/MR/...) z tagów DICOM,
- przeglądarka DICOM: przewijanie warstw, zoom, regulacja okna/poziomu (W/L),
- eksport listy obrazów do pliku XML (ścieżka, rozmiar, data),
- przechowywanie ścieżek do katalogów ze źródłami danych w bazie.

Pliki DICOM pozostają na dysku w oryginalnych katalogach; w bazie zapisywane są tylko metadane i ścieżki względne względem katalogu źródła.

2. Struktura kodu źródłowego

Cały kod aplikacji znajduje się w jednym pliku main.cpp. Konfiguracja budowania to CMakeLists.txt. Pozostałe pliki to ikona (app.ico, app.rc), przykładowe dane (data/) oraz skrypt uruchamiający (start_baza.bat).

```
SIM/
+-- main.cpp          - cała aplikacja (klasy + funkcje pomocnicze + main)
+-- CMakeLists.txt    - konfiguracja budowania (CMake)
+-- app.ico / app.rc  - ikona pliku wykonywalnego (Windows)
+-- assets/app.png    - źródłowa grafika ikony
+-- data/             - przykładowe dane DICOM (CT-00000, MRI-00000)
+-- start_baza.bat    - uruchamia serwer PostgreSQL + aplikację
+-- release/          - zbudowana aplikacja (exe + biblioteki DLL)
```

Logiczny podział main.cpp:

1. funkcje pomocnicze bazy/DICOM (wolne funkcje),
2. klasy interfejsu (dialogi i widżety),
3. okno główne MainWindow,
4. funkcja main().

3. Struktury danych (schemat bazy)

Schemat jest czteropoziomowy: pacjent -> badanie -> seria -> obraz, plus pomocnicza tabela źródeł danych.

```

patients (pacjent)
  id PK
  surname, name, birth_date, pesel, sex
  |
  | 1
  |
  | N
studies (badanie)
  id PK
  patient_id FK -> patients.id
  study_date, type, description
  |
  | 1
  |
  | N
series (seria)
  id PK
  study_id FK -> studies.id
  modality, description
  |
  | 1
  |
  | N
images (obraz)
  id PK
  series_id FK -> series.id
  source_id FK -> sources.id
  file_path      (ścieżka WZGLĘDNA do katalogu źródła)

sources (źródło danych)
  id PK
  nazwa
  sciezka  (UNIQUE - bezwzględna ścieżka katalogu na dysku)

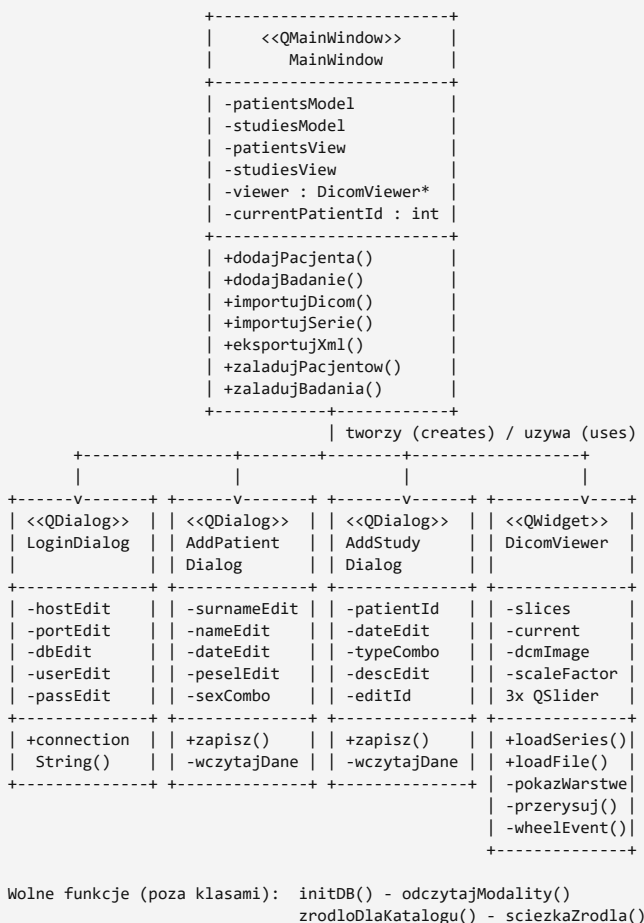
```

Pełna ścieżka pliku na dysku odtwarzana jest jako:
sources.sciezka + "/" + images.file_path.

Tabele tworzone są automatycznie przy starcie aplikacji (funkcja initDB())
przez CREATE TABLE IF NOT EXISTS. Klucze obce (REFERENCES) zapewniają
spójność między poziomami.

4. Klasy i diagram UML

Aplikacja opiera się na klasach Qt (QDialog, QWidget, QMainWindow).



LoginDialog (QDialog)

Okno logowania do bazy. Pola: serwer, port, baza, użytkownik, hasło.

Metoda `connectionString()` buduje tekst połączenia ODBC dla sterownika PostgreSQL.

AddPatientDialog (QDialog)

Formularz dodawania i edycji pacjenta (nazwisko, imię, data urodzenia, PESEL, płeć). Pole `editId` decyduje, czy to dodawanie (INSERT) czy edycja (UPDATE). Metody: `zapisz()`, `wczytajDane()`.

AddStudyDialog (QDialog)

Formularz dodawania i edycji badania (data, typ: RTG/CT/USG/MRI, opis). Badanie jest powiązane z wybranym pacjentem (`patientId`).

DicomViewer (QWidget)

Przeglądarka obrazów DICOM. Przechowuje listę ścieżek warstw (slices), aktualny indeks (current) oraz wczytany obraz `DicomImage` z DCMTK.

Obsługuje:

- `loadSeries()` / `loadFile()` - wczytanie serii lub pojedynczego pliku,
- `pokazWarstwe()` - przełączenie warstwy,
- `przerysuj()` - renderowanie obrazu z aktualnym oknem/poziomem i skalą,
- `wheelEvent()` - kółko myszy (warstwy) i Ctrl+kółko (zoom).

MainWindow (QMainWindow)

Okno główne. Po lewej lista pacjentów z filtrem, po prawej zakładki "Badania" i "Podgląd DICOM". Zawiera modele `QSqlQueryModel` dla pacjentów i badań. Obsługuje import, eksport, edycję i usuwanie przez menu pod prawym przyciskiem myszy.

Funkcje pomocnicze (wolne funkcje)

- `initDB()` - tworzy tabele bazy,
- `odczytajModality(path)` - odczyt tagu Modality z pliku DICOM (DCMTK),
- `zrodloDlaKatalogu(dir)` - znajduje lub dodaje źródło dla katalogu,
- `sciezkaZrodla(id)` - zwraca ścieżkę katalogu źródła po jego id.

5. Funkcje i metody (opis)

Funkcje pomocnicze (wolne funkcje)

Funkcja	Opis
<code>initDB()</code>	Tworzy tabele bazy (CREATE TABLE IF NOT EXISTS); zwraca bool.
<code>odczytajModality(path)</code>	Odczytuje tag Modality (CT/MR) z pliku DICOM.
<code>zrodloDlaKatalogu(dir)</code>	Znajduje źródło dla katalogu lub dodaje nowe; zwraca id.
<code>sciezkaZrodla(id)</code>	Zwraca bezwzględną ścieżkę źródła po jego id.

LoginDialog

Metoda	Opis
<code>LoginDialog(parent)</code>	Buduje okno logowania (serwer, port, baza, user, hasło).
<code>connectionString()</code>	Składa tekst połączenia ODBC (różny dla Windows/Linux).

AddPatientDialog

Metoda	Opis
<code>AddPatientDialog(parent, editId)</code>	Formularz pacjenta; <code>editId > 0</code> = edycja.
<code>zapisz()</code>	Waliduje i zapisuje pacjenta (INSERT/UPDATE).
<code>wczytajDane()</code>	Wczytuje dane pacjenta przy edycji.

AddStudyDialog

Metoda	Opis
<code>AddStudyDialog(patientId, parent, editId)</code>	Formularz badania dla pacjenta.
<code>zapisz()</code>	Zapisuje badanie (INSERT/UPDATE).
<code>wczytajDane()</code>	Wczytuje dane badania przy edycji.

DicomViewer

Metoda	Opis
<code>loadSeries(paths)</code>	Wczytuje serię (listę warstw) i pokazuje pierwszą.

loadFile(path)	Wczytuje pojedynczy plik jako serię z jedną warstwą.
pokazWarstwe(idx)	Przełącza i wczytuje warstwę o indeksie idx.
przerysuj()	Renderuje obraz z aktualnym oknem/poziomem i skalą.
wheelEvent(e)	Kółko = warstwy, Ctrl+kółko = zoom.

MainWindow

Metoda	Opis
onPatientClicked(i)	Klik pacjenta -> ładuje jego badania.
onStudyDoubleClicked(i)	Dwuklik badania -> ładuje obrazy do podglądu.
dodajPacjenta() / dodajBadanie()	Otwierają formularze dodawania.
importujDicom()	Import pojedynczego pliku DICOM do badania.
importujSerie()	Import całego folderu (serii) DICOM.
eksportujXml()	Eksport listy obrazów do pliku XML.
filtrujPacjentow(t)	Filtruje listę pacjentów po nazwisku.
patientMenu(p) / studyMenu(p)	Menu PPM: edycja / usuwanie.
usunPacjenta(id) / usunBadanie(id)	Usuwanie kaskadowe.
zaladujPacjentow(f) / zaladujBadania(id)	Ładują dane do tabel.

6. Algorytmy

Renderowanie DICOM (okno / poziom). Obraz DICOM ma wartości w dużym zakresie (np. jednostki Hounsfielda). Z suwaków pobierane są center (poziom) i width (okno), ustawiane przez DicomImage::setWindow(). Następnie getOutputData(8) zwraca 8-bitowe piksele, które kopiowane są wiersz po wierszu do QImage(Format_Grayscale8) i skalowane współczynnikiem scaleFactor.

Nawigacja po warstwach. Seria to lista plików slices. Kółko myszy zmienia indeks current (pokazWarstwe(current $\pm$ 1)); ten sam indeks ustawia pionowy suwak. Indeks jest przycinany do zakresu [0, liczba_warstw-1].

Wykrywanie modalności. Z pierwszego pliku serii odczytywany jest tag DCM_Modalitiy (DCMTK findAndGetOFString) - np. "MR" lub "CT".

Ścieżki względne i źródła. Przy imporcie katalog pliku porównywany jest z istniejącymi źródłami; jeśli żadne nie jest prefiksem ścieżki, dodawane jest nowe źródło. W tabeli images zapisywana jest ścieżka względna (QDir::relativeFilePath). Dzięki temu przeniesienie katalogu danych wymaga tylko zmiany wpisu w sources.

Eksport XML. Zapytanie łączy images z series, studies, patients i sources, odtwarza pełną ścieżkę, a dla każdego pliku odczytuje z dysku rozmiar i datę (QFileInfo). Wynik zapisywany jest przez QXmlStreamWriter.

Usuwanie kaskadowe. Usunięcie pacjenta kasuje najpierw obrazy, potem serie, badania, a na końcu pacjenta (zgodnie z zależnościami kluczy obcych).

7. Biblioteki

- Qt 6 (moduły Widgets, Sql) - interfejs graficzny, modele danych, połączenie z bazą (QSqlDatabase ze sterownikiem QODBC), zapis XML (QXmlStreamWriter). Licencja LGPL v3.
- DCMTK 3.6.8 - wczytywanie i dekodowanie plików DICOM (DicomImage, dekodery JPEG / JPEG-LS, odczyt tagów). Licencja BSD-like.
- PostgreSQL 16 - serwer bazy danych. Licencja PostgreSQL.
- psqLODBC (sterownik PostgreSQL ODBC) - warstwa komunikacji ODBC.

8. Kompilacja

System budowania: CMake + Ninja.

Windows (MinGW)

```
$env:Path = "C:\Qt\Tools\mingw1310_64\bin;" + $env:Path
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release `
  -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++ `
  -DCMAKE_PREFIX_PATH="C:\Qt\6.8.1\mingw_64" -DDCMTK_DIR="C:\dcmtdk\cmake"
cmake --build build
```

Po zbudowaniu biblioteki Qt zbierane są obok exe narzędziem windeployqt (dołącza m.in. sterownik sqldrivers\qsqlodbc.dll).

Ubuntu / Linux

Zależności:

```
sudo apt install build-essential cmake ninja-build \
  qt6-base-dev libqt6sql6-odbc libdcmtdk-dev \
  unixodbc odbc-postgresql postgresql
```

Budowanie:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Plik wykonywalny: build/BazaBadanObrazowych.

CMakeLists.txt szuka pakietów Qt6 i DCMTK przez find_package.

Na Linuxie nie trzeba podawać DCMTK_DIR - biblioteka jest w systemie (po apt install libdcmtdk-dev). Pliki tylko pod Windows (ikona, biblioteki sieciowe) są w warunku if(WIN32).

9. Instalacja w systemie

1. Zainstalować PostgreSQL 16 i utworzyć bazę medbaza.
2. Zainstalować sterownik psqLODBC (PostgreSQL Unicode x64) - z pakietu MSI z postgresql.org, wymaga uprawnień administratora.
3. Skopiować katalog release\ z aplikacją na komputer docelowy.
4. Uruchomić serwer bazy i aplikację (skrypt start_baza.bat).

Tabele tworzą się same przy pierwszym uruchomieniu. Dane importuje się z poziomu aplikacji (przyciski "Importuj DICOM" / "Importuj serie").

10. Konfiguracja

- Serwer PostgreSQL działa na porcie 5433 (pg_ctl ... -o "-p 5433").
- Dane logowania domyślne: serwer 127.0.0.1, baza medbaza, użytkownik postgres. Wpisuje się je w oknie logowania przy starcie.
- Tekst połączenia ODBC (klasa LoginDialog):
Windows: Driver={PostgreSQL Unicode(x64)};...
Linux: Driver={PostgreSQL Unicode};...
- Źródła danych (katalogi z plikami) dodają się automatycznie przy imporcie.